

halfedge：示例集设计文档

examples/

halfedge 库

2026-07-04

目录

1 示例总览	2
2 示例依赖关系	4
3 各示例说明	4
3.1 storage_basic: 底层存储 CRUD	4
3.2 build_mesh: 从顶点面索引构建网格	4
3.3 obj_io: OBJ 读写	4
3.4 traversal: 邻域遍历	5
3.5 topology_ops: 三大拓扑操作	5
3.6 extrude_face: 三角面挤出	5
3.7 geometry_query: 几何查询	5
3.8 laplacian_smooth: 拉普拉斯平滑	6
3.9 point_triangle_distance: 点到三角形距离	6
3.10 loop_subdivision: Loop 细分	6
3.11 property: OpenMesh 风格属性系统	6
3.12 validate: 拓扑自检	7
3.13 export_wgpu: wgpu 缓冲导出	7
3.14 icosphere: 球体生成与性质验证	7
3.15 engvis_viewer: engvis-renderer 交互式 3D 可视化	7
4 运行方式	9
4.1 运行单个示例	9
4.2 编译所有示例（不运行）	9
4.3 列出所有可用示例	9
4.4 运行 engvis_viewer 可视化示例	9
5 示例设计原则	10

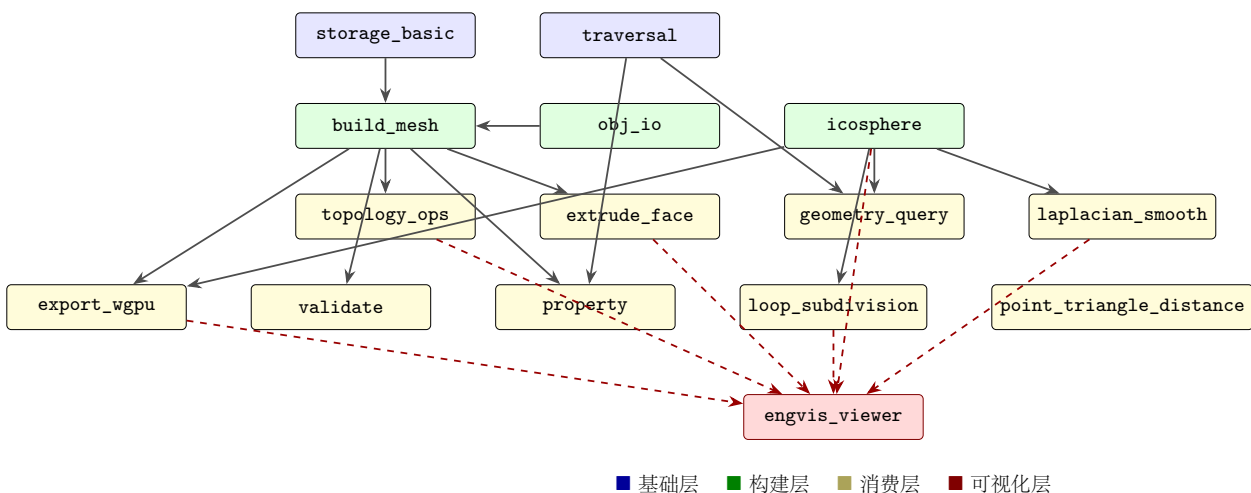
6 典型输出示例	10
6.1 icosphere 输出片段	10
6.2 topology_ops 输出片段	10
6.3 loop_subdivision 输出片段	11
6.4 extrude_face 输出片段	11
6.5 property 输出片段	11
6.6 engvis_viewer 输出片段	12

1 示例总览

`examples/` 目录下提供 15 个独立可运行的示例, 按功能划分, 每个示例聚焦一个模块的核心用法。前 14 个示例为纯文本输出, 无需额外依赖; 第 15 个 `engvis_viewer` 为 GPU 交互式 3D 可视化示例, 依赖 `engvis-renderer` (已声明为 `dev-dependency`)。所有示例通过 `cargo run --example <name>` 运行。

示例	模块	功能	运行命令
storage_basic	storage	顶点/半边/面 CRUD 与句柄有效性	cargo run --example storage_basic
build_mesh	io	从顶点 + 面索引构建完整网格	cargo run --example build_mesh
obj_io	io	OBJ 文本/文件读写往返	cargo run --example obj_io
traversal	traversal	5 迭代器 + 2 边界判定函数	cargo run --example traversal
topology_ops	topology_ops	split/flip/collapse 三大操作	cargo run --example topology_ops
extrude_face	topology_ops	单/批量三角面挤出形成棱柱	cargo run --example extrude_face
geometry_query	geometry	边长/面积/法向/最小内角/顶点法向	cargo run --example geometry_query
laplacian_smooth	geometry	拉普拉斯平滑 (顶点级 + 整网级)	cargo run --example laplacian_smooth
point_triangle_distance	geometry	点到三角形 7-Voronoi 区域距离	cargo run --example point_triangle_distance
loop_subdivision	subdiv	Loop 细分规模/度数/Euler 验证	cargo run --example loop_subdivision
property	property	OpenMesh 风格属性系统全演示	cargo run --example property
validate	validate	拓扑自检: 干净通过 + 破坏检测	cargo run --example validate
export_wgpu	export	导出 wgpu 顶点/索引缓冲	cargo run --example export_wgpu
icosphere	test_util	icosphere 生成与性质验证	cargo run --example icosphere
engvis_viewer		engvis-renderer 交互式 3D 可视化	cargo run --example engvis_viewer

2 示例依赖关系



3 各示例说明

3.1 storage_basic: 底层存储 CRUD

演示 MeshStorage 的增删改查与 slotmap 的 ABA 安全保证。关键点:

- 创建 3 顶点 + 3 半边 + 1 面, 手动设置 next/prev/face 环;
- 删除顶点后句柄立即失效, contains_vertex 返回 false;
- 槽位复用后旧句柄不复活 (版本号机制)。

3.2 build_mesh: 从顶点面索引构建网格

演示 build_mesh_from_vertices_faces, 自动构建 twin/next/prev/边界环。关键点:

- 输入 4 顶点 + 2 三角形 (共享 1 边), 输出 10 半边 (2 内部 + 8 边界);
- 用 validate_topology 验证构建结果;
- 用 is_boundary_edge 区分内部边与边界边。

3.3 obj_io: OBJ 读写

演示 parse_obj / format_obj / save_obj / load_obj。关键点:

- 解析四面体 OBJ 文本, 要求面绕向一致;
- 序列化回 OBJ 文本, 文件 IO 往返一致;
- 支持 v/vt/vn 格式与负索引。

3.4 traversal: 邻域遍历

演示 5 个迭代器 + 2 个边界判定函数。关键点:

- `VertexRing`: 顶点的 outgoing 半边环 (CCW 顺序);
- `VertexAdjacentVerts` / `VertexAdjacentFaces`: 邻居顶点 / 邻接面;
- `FaceHalfEdges` / `FaceVertices`: 面边界环;
- 迭代期不持有借用, 可自由 `&mut mesh`。

3.5 topology_ops: 三大拓扑操作

演示 `split_edge` / `flip_edge` / `collapse_edge`。关键点:

- `split`: 顶点 +1, 面 +2 (内部边) 或 +1 (边界边);
- `flip`: 顶点/面/半边数不变, 仅重连 4 条半边;
- `collapse`: 顶点 -1, 面 -2, 半边 -6;
- 翻转边界边返回 `FlipOnBoundaryEdge` 错误。

3.6 extrude_face: 三角面挤出

演示 `extrude_face` / `extrude_faces` 将三角面沿给定方向 `offset` 挤出形成闭合三棱柱。关键点:

- 单面挤出: 新增 3 顶点 + 7 面 (1 反向顶面 + 6 侧面三角形), $Euler = 2$;
- 不同 `offset` 方向 (沿法向、斜向、反向) 都能正确闭合;
- 零向量 `offset` 返回 `DegenerateTriangle`;
- 与三角形共面的 `offset` (侧面面积 $< 10^{-12}$) 同样报退化;
- 批量挤出 `extrude_faces` 对不相交三角形集合一次性挤出;
- 挤出后所有面边界环长度均为 3, `validate_topology` 通过。

3.7 geometry_query: 几何查询

演示 5 个查询函数: 边长、面积、法向、最小内角、顶点法向。关键点:

- `icosphere(0)` 所有面为等边三角形, 最小内角 = 60° ;
- 总面积 $\approx 4\pi$ (单位球面面积);
- 顶点法向与顶点位置平行 (球面性质)。

3.8 laplacian_smooth: 拉普拉斯平滑

演示 `laplacian_smooth_vertex` 与 `laplacian_smooth_mesh`。关键点:

- 单顶点: 新位置 = 邻居位置算术平均;
- 整网: $\lambda = 0.5$, 5 次迭代, 重心偏移 $< 10^{-15}$ (保留重心);
- 平滑使球面收缩 (顶点偏离单位球面);
- $\lambda = 0$ 时网格不变。

3.9 point_triangle_distance: 点到三角形距离

演示 Ericson 7-Voronoi 区域算法。关键点:

- 面域: 投影在三角形内部, 距离 = 垂直高度;
- 顶点域 $V_A/V_B/V_C$: 距离到对应顶点;
- 边域 $E_{AB}/E_{AC}/E_{BC}$: 距离到边投影;
- 退化三角形 (共线) 退化为三顶点最近距离。

3.10 loop_subdivision: Loop 细分

演示 `subdiv::loop_subdivide` 在 icosphere 上的光滑细分效果。关键点:

- 单次细分规模: $V' = V + E$, $F' = 4F$ (icosphere(1) $42 \rightarrow 162$ 顶点, $80 \rightarrow 320$ 面);
- 连续多次细分保持 `check_topology` 通过;
- 度数分布: 原始 12 顶点保持度数 = 5, 新增边中点度数 = 6 (正则性质);
- Euler 示性数 $V - E + F = 2$ 在每次细分后保持;
- Loop 细分是逼近型细分, 单位球面偏差随细分次数减小;
- `validate_topology` 完整校验无错误。

3.11 property: OpenMesh 风格属性系统

演示 `property::MeshProperties` 与 `PropertyHandle<T>` 的核心用法。关键点:

- 注册 f64 顶点权重属性, `set/get_vertex_prop` 读写;
- 类型擦除: 同一容器并存 f64 / i32 / String / bool 多种属性;
- 加权拉普拉斯平滑: 北极顶点权重 5.0, 其余 1.0, 验证等权与加权位移;
- 半边属性: 缓存所有半边边长, 统计 min/max/avg;
- 面属性: 缓存面法向, 与面心方向一致;
- `remove_vertex_with_props` 删除顶点时同步清理属性;
- newtype 模式: 同底层 f64 通过 `Weight/Temperature` 注册两个互不冲突的属性。

3.12 validate: 拓扑自检

演示 `validate_topology` 的完整校验。关键点:

- 干净 `icosphere(1)` 通过校验 (0 错误);
- 人为破坏 `twin` 互指 → 检测 `TwinMismatch + SelfLoopHalfEdge`;
- 悬空顶点引用 → `HalfEdgeDanglingVertex`;
- 共线三角形 → `DegenerateFace`;
- 入口 `origin` 不匹配 → `VertexHalfEdgeInconsistent`。

3.13 export_wgpu: wgpu 缓冲导出

演示 `mesh_to_vertex_index_buffers`。关键点:

- 输出 `Vec<f32; 3>` 顶点 + `Vec<u32>` 索引;
- `icosphere(1)`: 42 顶点 (504 字节) + 240 索引 (960 字节);
- 索引范围合法 (`max index < vertex count`);
- CCW 朝向: 法向 · 重心 > 0 (朝外)。

3.14 icosphere: 球体生成与性质验证

演示 `build_icosphere` 在不同细分级别的网格性质。关键点:

- 规模公式: $V_n = 10 \cdot 4^n + 2$, $F_n = 20 \cdot 4^n$, $E_n = 30 \cdot 4^n$;
- 所有顶点在单位球面上 (偏差 < 10^{-15});
- 所有面法向朝外;
- `icosahedron(0)` 的 12 个顶点度数均为 5;
- Euler 公式 $V - E + F = 2$ 成立。

3.15 engvis_viewer: engvis-renderer 交互式 3D 可视化

前 14 个示例均通过 `println!` 输出文本结果, 缺乏直观的几何观察。`engvis_viewer` 借助 `engvis-renderer` (基于 `wgpu 27 / egui 0.33 / winit 0.30`) 提供 GPU 加速的 3D 交互式可视化, 将 `halfedge` 的网格操作结果直接渲染为可旋转、可缩放的立体图形。

依赖声明 `engvis-renderer` 仅作为 `dev-dependency` 引入, 不影响发布到 `crates.io` 的库本身:

```
1 [dev-dependencies]
2 egui                = "0.33"
3 engvis-core         = "0.1"
4 engvis-renderer     = "0.1"
5 winit               = "0.30"
```

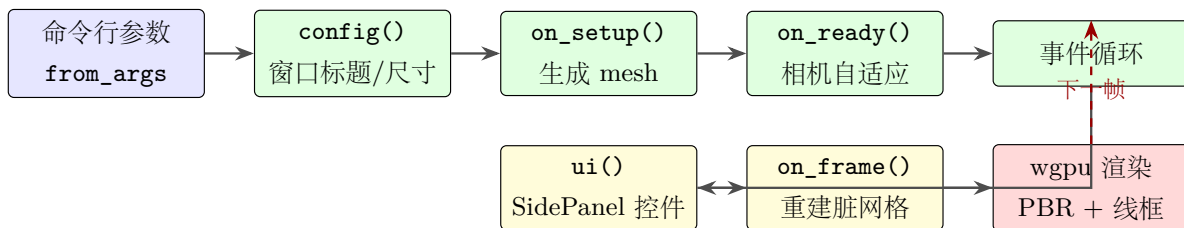
支持的 5 种操作 `engvis_viewer` 通过命令行参数或 UI 面板切换 5 种网格操作，覆盖 `halfedge` 主要模块：

操作	命令行	调用模块
Icosphere 生成	<code>icosphere [n]</code>	<code>test_util::build_icosphere</code>
曲面细分	<code>subdivision <loop cc sqrt3> n</code>	<code>subdiv::{loop,catmull_clark,sqrt3}_subdivide</code>
面挤出	<code>extrude</code>	<code>primitives::build_cube</code> + <code>topology_ops::extrude_face</code>
Laplacian 平滑	<code>smooth [iters]</code>	<code>primitives::build_uv_sphere</code> + <code>geometry::laplacian_smooth_mesh</code>
拓扑操作	<code>topology <split flip collapse></code>	<code>topology_ops::{split,flip,collapse}_edge</code>

halfedge → engvis 转换 `halfedge` 的 `MeshStorage` 与 `engvis` 的 GPU Mesh 之间通过 `mesh_to_vertex_index_b` 桥接，输出与 `wgpu` 兼容的 $(\text{Vec}\langle\text{f32};3\rangle, \text{Vec}\langle\text{u32}\rangle)$ ，再由 `Mesh::from_triangles` 自动去重、修复绕向、计算平滑法向：

$\text{MeshStorage} \xrightarrow{\text{mesh_to_vertex_index_buffers}} (\mathbf{V}, \mathbf{I}) \xrightarrow{\text{from_triangles}} \text{engvis Mesh}$

渲染流水线 `EngvisApp` trait 提供声明式生命周期，`viewer` 在各阶段完成不同任务：



UI 控件 右侧 `egui::SidePanel` 提供以下交互：

- **操作类型**：5 个 `selectable_label` 单选切换；
- **参数调整**：Slider（细分次数 $n \in [0, 4]$ 、平滑迭代 $\in [0, 50]$ ）、ComboBox（细分类型、拓扑操作）；
- **网格统计**：实时显示 V 、 E 、 F 与 Euler 示性数 $V - E + F$ ；
- **渲染选项**：曲面/网格地面/线框/顶点四个开关，分别绑定 `RenderState` 的 `show_surface` / `show_grid` / `edge_opts.enabled` / `vertex_opts.enabled`。

脏标记重建 为避免每帧重建网格造成卡顿，采用 `dirty` 标记：UI 修改参数时置 `dirty = true`，`on_frame` 检测到脏位才重新调用 `generate_mesh` 并替换场景中的第一个 `mesh`，重置 `scene_dirty` 触发 GPU 上传。

关键点

- 5 种操作覆盖 `test_util / subdiv / topology_ops / geometry / primitives / export` 六大模块；
- `Mesh::from_triangles` 自动完成顶点去重、绕向修复、平滑法向计算；
- 借用检查器冲突通过先收集 `face_ids / halfedge_ids` 到 `Vec` 再消费的模式解决；
- 窗口标题随操作类型动态变化(`format!("halfedge viewer {}", self.operation.label())`);
- 相机 `fit_to_scene` 在 `on_ready` 阶段自适应场景包围盒。

4 运行方式

4.1 运行单个示例

```
1 cargo run --example storage_basic
2 cargo run --example icosphere
```

4.2 编译所有示例（不运行）

```
1 cargo build --examples
```

4.3 列出所有可用示例

```
1 cargo run --example # Tab
```

4.4 运行 `engvis_viewer` 可视化示例

`engvis_viewer` 接受一个可选的子命令参数指定初始操作，未传时默认 `icosphere 1`:

```
1 # 1 icosphere
2 cargo run --example engvis_viewer
3
4 # 2 icosphere
5 cargo run --example engvis_viewer -- icosphere 2
6
7 # Loop 2 cc / catmull_clark / sqrt3
8 cargo run --example engvis_viewer -- subdivision loop 2
9
10 # cube (+Y )
11 cargo run --example engvis_viewer -- extrude
12
13 # Laplacian 10
14 cargo run --example engvis_viewer -- smooth 10
15
16 # split / flip / collapse
17 cargo run --example engvis_viewer -- topology split
```

启动后窗口右侧 UI 面板可实时切换 5 种操作与参数，无需重启程序。

5 示例设计原则

1. **自包含**：每个示例独立可运行，不依赖其他示例；
2. **聚焦单一功能**：每个示例只演示一个模块的核心 API；
3. **可观测输出**：前 14 个示例用 `println!` 打印关键数据，便于直观验证；
4. **使用真实网格**：大多数示例以 `build_icosphere` 作为测试网格，保证拓扑合法、几何有意义；
5. **包含错误处理**：`topology_ops` 与 `validate` 示例演示了错误路径（翻转边界边、检测破坏）；
6. **可视化增强**：`engvis_viewer` 借助 `engvis-renderer` 提供 GPU 加速 3D 交互式可视化，将文本输出升级为可旋转、可缩放的立体图形，实时切换 5 种网格操作与参数。

6 典型输出示例

6.1 icosphere 输出片段

```
1 === icosphere          ===
2
3 [
4     n
5     0      12      20      60      OK
6     1      42      80     240      OK
7     2     162     320     960      OK
8     3     642    1280    3840      OK
9     4    2562    5120   15360      OK
10
11 [  1:      ] icosphere(2)          = 2.22e-16
12 [  5: Euler] n=0: V-E+F = 12-30+20 = 2      2
```

6.2 topology_ops 输出片段

```
1 [split_edge]          VertexId(13v1)
2     12 → 13
3     20 → 22
4     validate_mesh: Ok(())
5     check_topology: OK
6
7 [collapse_edge]        VertexId(13v1)
8     12 → 11
9     20 → 18
10    60 → 54
11
12          → Some("HalfEdgeId(1v1)")
```

6.3 loop_subdivision 输出片段

```
1 [1]      Loop
2      icosphere(1): V=   42 F=   80 E=  120 HE=  240
3              : V=  162 F=  320 E=  480 HE=  960
4      V'=V+E (42+120=162) F'=4F (4*80=320)
5      OK
6
7 [4] Euler      V - E + F
8      0x: V-E+F = 12-30+20 = 2      2
9      1x: V-E+F = 42-120+80 = 2      2
10     2x: V-E+F = 162-480+320 = 2      2
11     3x: V-E+F = 642-1920+1280 = 2      2
```

6.4 extrude_face 输出片段

```
1 [1]
2      : V=   3 F=   1 E=   3 HE=   6
3      : V=   6 F=   8 E=  12 HE=  24
4      = 7      7 1      + 6
5      OK
6 Euler      V-E+F = 2      2
7
8 [3]      /      offset
9      offset DegenerateTriangle
10 x      offset DegenerateTriangle
```

6.5 property 输出片段

```
1 ===== 1.      =====
2      f64      42
3 vertex_prop_type_count = 1
4
5 ===== 2.      =====
6      i32 / String / bool
7 vertex_prop_type_count = 4
8
9 ===== 4.      =====
10     240      halfedge_prop_type_count = 1
11     min=0.6071, max=0.7174, avg=0.6760 ( 240      )
12
13 ===== 6.      =====
14
15 vVertexId(1v1) exists=true, weight=Some(1.0)
16
17 vVertexId(1v1) exists=false, weight=None
```

6.6 engvis_viewer 输出片段

启动时控制台打印操作类型与提示，剩余信息显示在窗口右侧 UI 面板：

```
1 === halfedge engvis viewer ===  
2 : Icosphere (n=1)  
3 : UI
```

UI 面板中的网格统计区会实时显示当前网格的拓扑量（随操作参数变化）：

```
1 halfedge viewer  
2 -----  
3  
4 [ ] icosphere  
5 [ ] subdivision  
6 [ ] extrude face  
7 [ ] smooth  
8 [*] topology  
9  
10  
11 V = 12  
12 E = 30  
13 F = 20  
14 Euler V-E+F = 2  
15  
16  
17 [x]  
18 [ ]  
19 [x]  
20 [ ]  
21  
22 FPS: 60.0
```